SELENIUM PYTHON AUTOMATION

MODULE 2 LAB WORKBOOK

Assignments 7–9: Page Object Model, Data-Driven Testing and PyTest Integration

Assignment	Focus	Evidence
7	Page Object Model (POM)	POM login result + execution output
8	Data-Driven Automation (DDT)	Five credential cases executed
9	PyTest Integration	PyTest terminal execution: PASSED

Module scope

The course curriculum places Assignments 7–9 in the framework-oriented part of the Selenium Python automation module. The documented tasks cover restructuring Selenium scripts using POM, executing multiple login combinations from external test data, and moving Selenium execution into PyTest with fixtures and HTML-reporting capability.

Tools and concepts

Python, Selenium WebDriver, Microsoft Edge, Page Object Model, external CSV test data, PyTest fixtures, assertions and terminal-based test execution.

Experiment 7 — Page Object Model (POM)

Problem statement: Restructure the working Selenium login automation into a Page Object Model design, keeping page interaction logic separate from the test workflow and validation.

Objective: Create a LoginPage class containing the login-page interaction methods, then use that class from the test script. The curriculum specifically calls for separate page classes containing locator definitions and UI methods, with assertions kept outside the page object.

Implementation steps

1. Create a LoginPage class and store the WebDriver instance. 2. Implement methods for entering the username, entering the password and clicking Login. 3. Instantiate LoginPage from the test script. 4. Perform the login workflow through page methods. 5. Validate that the resulting URL contains **/inventory.html**.

Representative structure

```
LoginPage(driver) → enter_username() → enter_password() → click_login()
Test script → create LoginPage → execute workflow → assert /inventory.html
```

Output evidence

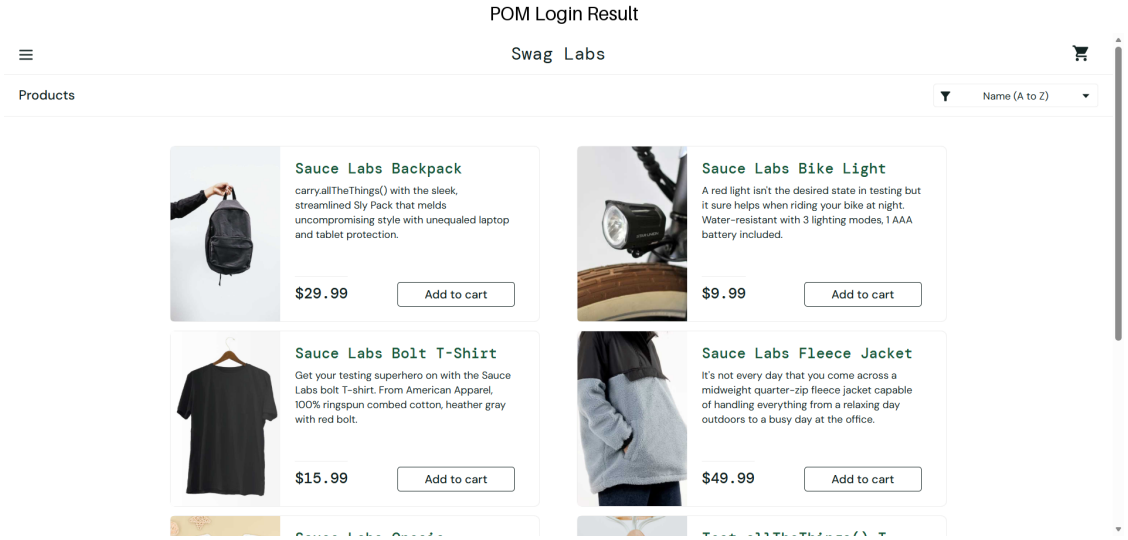


Figure 7.1 — POM Login Result

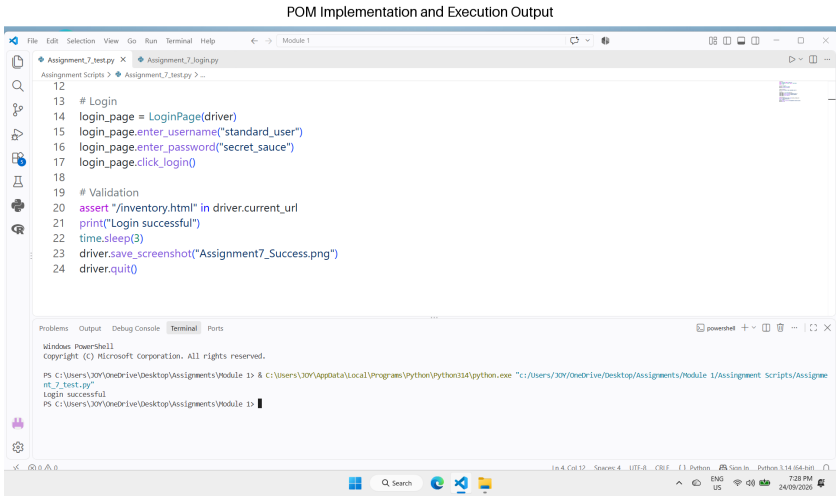


Figure 7.2 — POM Execution Output

Experiment 8 — Data-Driven Automation (DDT)

Problem statement: Build a login automation that reads multiple test cases from an external source and executes correct and incorrect credential combinations, validating the corresponding result for each case.

Objective: Use external CSV data to avoid hard-coding a separate test script for every credential combination. The curriculum explicitly permits an external Excel source or local JSON/CSV source and requires validation of the expected login outcome.

Test data and execution

The completed CSV contained five cases: one successful login using **standard_user / secret_sauce**, followed by four error cases covering a locked-out account, an incorrect password, an incorrect username and a standard user with an incorrect password.

Implementation flow

```
Read CSV → iterate through rows → open SauceDemo → enter username/password from current row → click Login → if expected = success, assert /inventory.html → otherwise verify the displayed validation error.
```

Observed execution

The execution output shows all five data rows processed successfully. The valid case produced “Login successful”, while each invalid case produced “Validation error displayed”.

Data-Driven Login Execution Output

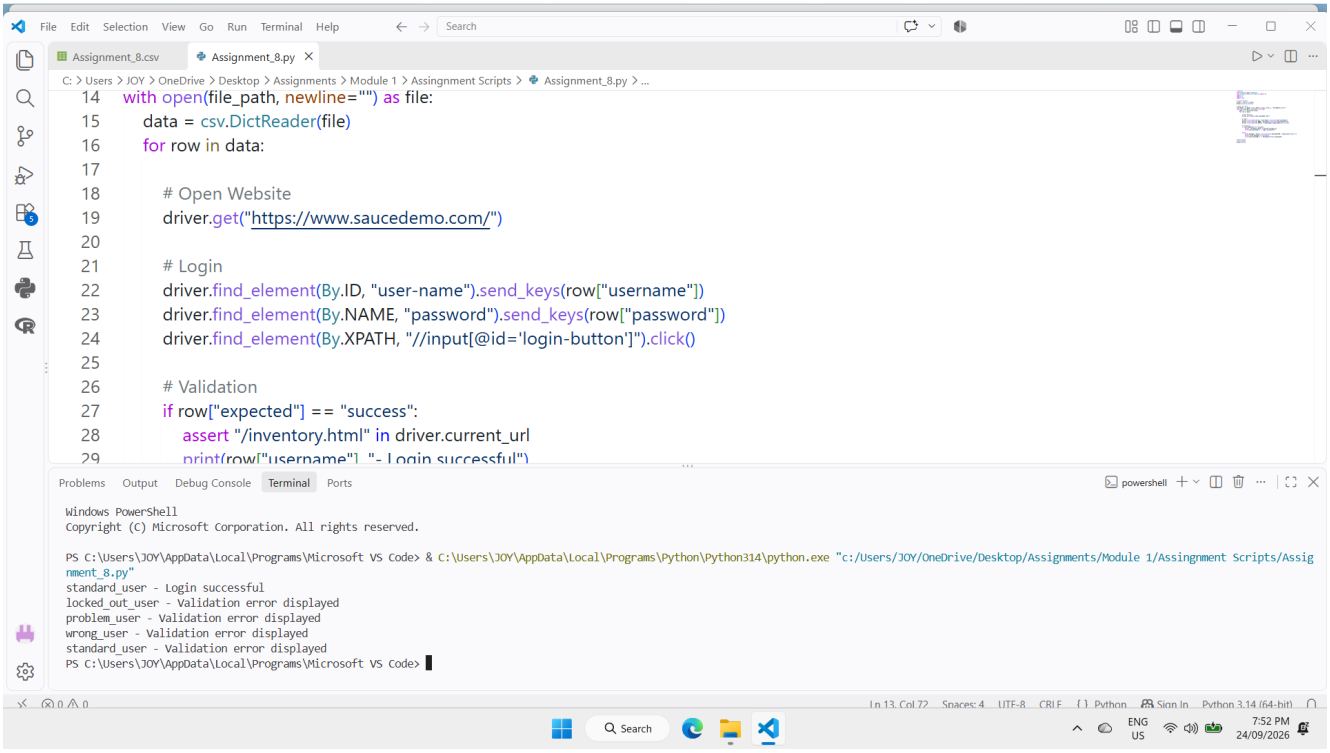


Figure 8.1 — Data-Driven Login Execution Output

Experiment 9 — PyTest Integration

Problem statement: Move the Selenium login test into the PyTest framework and use a fixture to manage browser initialization and teardown. The curriculum also introduces PyTest HTML reporting and failure screenshots as part of the framework capability.

Objective: Demonstrate a PyTest-based Selenium test with clean browser lifecycle management and terminal execution.

Implementation steps

1. Import PyTest and Selenium. 2. Define a **@pytest.fixture** that starts Microsoft Edge, maximizes the browser and yields the driver. 3. Close the driver after the test using fixture teardown. 4. Create the login test using the fixture. 5. Execute from the terminal with the PyTest command and verbose option.

Framework pattern

```
@pytest.fixture → start Edge → yield driver → quit browser
test_login(driver) → open page → login → assert /inventory.html
```

Execution result

The terminal evidence shows PyTest discovering one test item and completing **Assignment_9.py::test_login PASSED [100%]**. The run also shows the installed PyTest HTML plugin in the environment, matching the curriculum's reporting component.

PyTest Execution Output

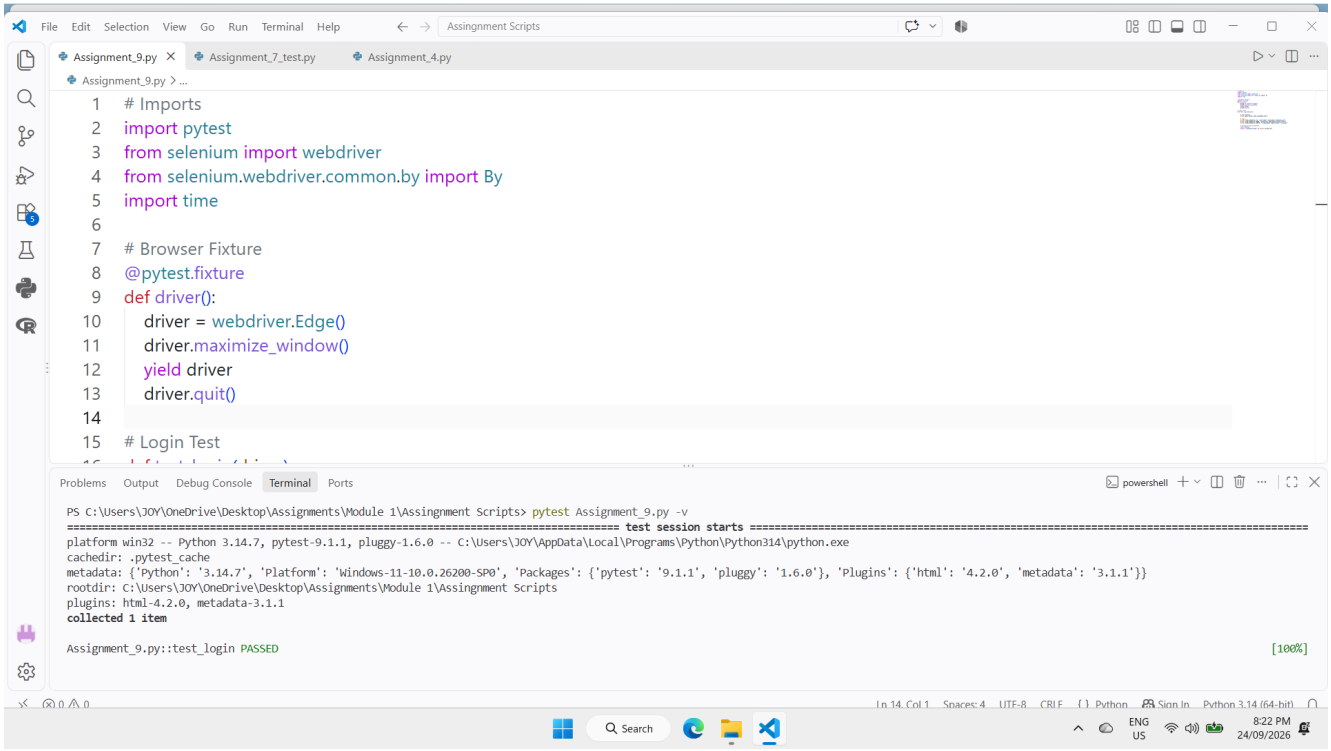


Figure 9.1 — PyTest Execution Output

Module 2 — Result, Observations and Conclusion

Overall result: Assignments 7–9 demonstrate the progression from a working Selenium script toward a maintainable test-automation structure: first separating page interaction logic through POM, then externalizing test data for repeated execution, and finally integrating the test into PyTest for framework-based execution.

Observations

• POM separates UI interaction methods from the test workflow and validation.
• DDT allows the same login workflow to process multiple credential combinations from CSV data.
• PyTest fixtures provide a reusable browser setup/teardown mechanism.
• Assertions remain part of the test validation rather than being hidden inside the page object.
• Terminal execution provides clear pass/fail evidence for the automation run.

Learning outcome

On completion of Module 2, the automation has moved beyond individual Selenium exercises into basic framework practices: reusable page objects, externalized test data, fixture-based lifecycle management and test-runner execution. These practices directly correspond to the curriculum's framework-oriented assignments.

Conclusion

Module 2 successfully demonstrates POM, data-driven Selenium automation and PyTest integration. The execution screenshots provide labelled evidence for each assignment, while the implementation follows the assignment objectives documented in the Python Automation curriculum.

Source basis

Course curriculum: *CoE-TOC-PYTHON_AUTOMATION.xlsx*, Python Automation ToC — Assignments 7, 8 and 9. Execution evidence: labelled screenshots supplied for Module 2. Implementation evidence: completed assignment scripts/work discussed for the module.